

**LAPORAN PRAKTIKUM BSD 2 HARI 6
(INTEGRASI MONGODB PYTHON)**



**MUHAMMAD ZAIDAN HABIBI
(225443020)**



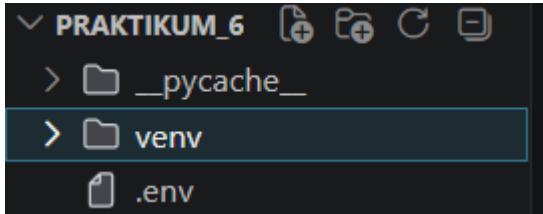
**1AEC-1
TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI
POLITEKNIK MANUFAKTUR BANDUNG
TAHUN AJARAN 2025/2026**

PENDAHULUAN

Pada praktikum ini, kami berfokus pada pembahasan mengenai proses memasukkan data dummy yang sebelumnya telah dibuat dalam format JSON, namun kali ini dilakukan melalui bahasa pemrograman Python untuk meningkatkan fleksibilitas dan otomatisasi dalam pengolahan data. Pendekatan ini memungkinkan integrasi data dummy ke dalam database MongoDB secara lebih dinamis dan terstruktur, di mana Python berperan sebagai perantara yang mengelola koneksi ke database, membaca data JSON, serta melakukan operasi insert ke dalam koleksi yang dituju. Selain itu, penggunaan Python juga memberikan kemudahan dalam melakukan manipulasi data sebelum disimpan, seperti penyesuaian format, validasi, maupun penambahan field tertentu sesuai kebutuhan skenario praktikum. Dengan demikian, metode ini tidak hanya mensimulasikan kondisi lingkungan produksi yang lebih realistis, tetapi juga memperkenalkan konsep integrasi antar teknologi yang umum digunakan dalam pengembangan sistem berbasis data.

PRAKTIKUM TERBIMBING

Tahap 1 – Setup Lingkungan



- Buka terminal dengan menekan shortcut `ctrl+shift+``, lalu jalankan `python -m venv venv`. Tujuannya agar virtual environment mengisolasi dependensi proyek ini dari proyek lain, mencegah konflik versi pustaka.
- Buat folder praktikum 6, lalu buat file `.env` dengan isi

```
MONGO_URI=mongodb://localhost:27017
DB_NAME=praktikum6
```

File ini akan memisahkan konfigurasi sensitif dari kode. Dengan memanggil `load_dotenv()` nanti, variabel-variabel ini akan tersedia di seluruh aplikasi tanpa kita hardcode.

- Buat file `koneksi.py` dan tuliskan kode berikut:

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()

client = MongoClient(os.getenv("MONGO_URI"), serverSelectionTimeoutMS=5000)

try:
    client.admin.command('ping')
    print("Koneksi MongoDB berhasil!")
except Exception as e:
```

```
print("Gagal terhubung:", e)

finally:

    client.close()
```

load_dotenv() membaca file .env. os.getenv() mengambil URI sehingga kode tidak mengandung alamat server secara langsung. Parameter serverSelectionTimeoutMS mencegah program menggantung jika server mati. Blok try-except menangani kegagalan koneksi dengan elegan, dan client.close() memastikan sumber daya dibebaskan. Jalankan skrip ini. Jika muncul "Koneksi MongoDB berhasil!", Anda siap melanjutkan.

Tahap 2 – Operasi Dasar CRUD

Melakukan operasi insert, read, update, dan delete pada koleksi sensor di MongoDB menggunakan PyMongo, serta memverifikasi hasilnya melalui MongoDB Compass.

Buat file crud_sensor.py dan tuliskan kode berikut:

```
from dotenv import load_dotenv
from pymongo import MongoClient
from datetime import datetime, timedelta
import random, os

# Load environment variable dan koneksi
load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

# INSERT MANY
data = []
for i in range(10):
    doc = {
        "mesin": f"CNC-{random.randint(1,3):02d}",
```

```

        "suhu": round(random.uniform(60, 100), 2),
        "getaran": round(random.uniform(0.1, 0.5), 2),
        "timestamp": datetime.utcnow() - timedelta(minutes=i*5),
        "status": "normal"
    }
    data.append(doc)
result = collection.insert_many(data)
print(f"Tersimpan {len(result.inserted_ids)} dokumen")

# READ
cursor = collection.find(
    {"mesin": "CNC-01"},
    {"_id": 0, "suhu": 1, "timestamp": 1}
).sort("timestamp", -1).limit(5)
print("Data CNC-01 terbaru:")
for doc in cursor:
    print(doc)

# UPDATE
update_result = collection.update_many(
    {"suhu": {"$gt": 90}},
    {"$set": {"status": "maintenance"}}
)
print(f"Update: {update_result.modified_count} dokumen diubah")

# DELETE
satu_jam_lalu = datetime.utcnow() - timedelta(hours=1)
delete_result = collection.delete_many({"timestamp": {"$lt": satu_jam_lalu}})
print(f>Delete: {delete_result.deleted_count} dokumen dihapus")

client.close()

```

Penjelasan Tiap Operasi

- **Insert Many** Sebanyak 10 dokumen dimasukkan sekaligus menggunakan `insert_many()`. Setiap dokumen memiliki field mesin, suhu, getaran, timestamp, dan status. Timestamp dibuat mundur 5 menit per iterasi agar data lebih realistis dan tidak semuanya berada pada waktu yang sama. Nilai sensor divariasikan menggunakan `random.randint` dan `random.uniform`.
- **Read Query** dilakukan dengan filter `{"mesin": "CNC-01"}` untuk mengambil data spesifik. Projection `{"_id": 0, "suhu": 1, "timestamp": 1}` membatasi field yang dikembalikan agar output lebih bersih. Data diurutkan secara descending berdasarkan timestamp dan dibatasi 5 dokumen teratas.
- **Update** Operasi `update_many` digunakan untuk mengubah field status menjadi "maintenance" pada semua dokumen yang memiliki nilai suhu di atas 90. Operator `$gt` (greater than) digunakan sebagai filter kondisi kritis, sementara `$set` memperbarui field tanpa menghapus field lainnya.
- **Delete** Data yang sudah lebih tua dari 1 jam dihapus menggunakan `delete_many`. Waktu batas dihitung dengan `datetime.utcnow() - timedelta(hours=1)`, dan operator `$lt` (less than) digunakan untuk mencocokkan dokumen yang timestampnya sebelum batas tersebut.

Tahap 3 – Agregasi via Python

Buat file `agregasi.py` yang menghitung rata-rata suhu per mesin, getaran maksimum, dan jumlah data, lalu mengurutkannya.

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]
```

```

pipeline = [
    {"$group": {
        "_id": "$mesin",
        "rata_suhu": {"$avg": "$suhu"},
        "max_getaran": {"$max": "$getaran"},
        "count": {"$sum": 1}
    }},
    {"$sort": {"rata_suhu": -1}}
]

results = collection.aggregate(pipeline)
print("Rata-rata suhu per mesin:")
for doc in results:
    print(f"{doc['_id']} -> rata:{doc['rata_suhu']:.2f}°C, max getaran:{doc['max_getaran']},  
jumlah data:{doc['count']}")
client.close()

```

Agregasi dengan operator \$group digunakan untuk mengelompokkan data berdasarkan mesin, di mana field mesin dijadikan sebagai _id pada hasil output. Operator \$avg dan \$max berfungsi untuk menghitung nilai rata-rata serta nilai maksimum, sementara \$sum: 1 digunakan untuk memperoleh jumlah data dalam tiap kelompok. Selanjutnya, hasil agregasi diurutkan secara menurun berdasarkan nilai rata_suhu menggunakan \$sort. Pendekatan ini menghasilkan ringkasan performa termal tiap mesin secara ringkas, sekaligus meningkatkan efisiensi karena pemrosesan dilakukan langsung di server MongoDB sehingga meminimalkan kebutuhan transfer data.

Tahap 4 – Analisis dengan Pandas

Buat file analisis.py yang mengambil seluruh data dari koleksi sensor, memuatnya ke DataFrame, melakukan analisis deskriptif, resampling, plotting, dan ekspor ke CSV.

- Import library pandas as pd dan matplotlib.pyplot as plt.
- Buat file analisis.py dan tuliskan kode sebagai berikut:

```
import os
```

```

from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

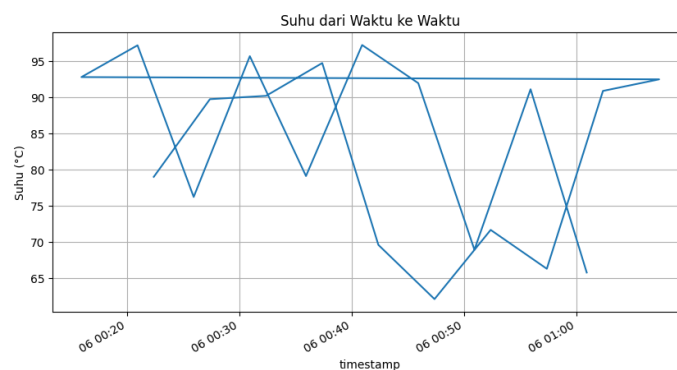
pipeline = [
    {"$group": {
        "_id": "$mesin",
        "rata_suhu": {"$avg": "$suhu"},
        "max_getaran": {"$max": "$getaran"},
        "count": {"$sum": 1}
    }},
    {"$sort": {"rata_suhu": -1}}
]

results = collection.aggregate(pipeline)
print("Rata-rata suhu per mesin:")
for doc in results:
    print(f"{doc['_id']} -> rata:{doc['rata_suhu']:.2f}°C, max  
getaran:{doc['max_getaran']}, jumlah data:{doc['count']}")

client.close()

```

Output:



Penjelasan:

- **Import dan Koneksi** Library pandas digunakan untuk manipulasi data tabular, matplotlib.pyplot untuk visualisasi grafik, dan collection

diambil dari `crud_sensor.py` sebagai sumber data MongoDB yang sudah terhubung.

- **Mengambil Data dari MongoDB** Seluruh dokumen pada koleksi `sensor` diambil menggunakan `collection.find({}, {"_id": 0})`. Parameter `{}` berarti tanpa filter sehingga semua dokumen dikembalikan, sedangkan `{"_id": 0}` mengecualikan field `_id` agar DataFrame lebih bersih. Hasil cursor dikonversi menjadi list of dict lalu dimuat ke DataFrame.
- **Konversi Timestamp dan Set Index** Kolom timestamp dikonversi dari tipe string atau datetime Python menjadi tipe `datetime64` milik pandas menggunakan `pd.to_datetime()`. Kolom tersebut kemudian dijadikan index DataFrame dengan `set_index()`. Index bertipe datetime wajib ada agar operasi time series seperti resampling dapat dilakukan.
- **Resampling** `df.resample('10min').mean(numeric_only=True)` mengelompokkan data ke dalam interval 10 menit dan menghitung rata-rata tiap interval. Parameter `numeric_only=True` ditambahkan agar kolom bertipe string seperti mesin dan status tidak ikut diproses, sehingga tidak menimbulkan error. Hasilnya disimpan ke variabel `resampled` dan 5 baris pertama ditampilkan dengan `head()`.
- **Plot Grafik** Grafik garis dibuat dari kolom suhu pada DataFrame original menggunakan `df['suhu'].plot()`. Ukuran figure diatur (10, 5) agar proporsional. Label sumbu Y, judul, dan grid ditambahkan untuk keterbacaan. Grafik disimpan ke file `suhu_plot.png` menggunakan `savefig()` sebelum `show()` agar file tersimpan dengan benar jika urutannya terbalik, file yang tersimpan bisa kosong.
- **Ekspor ke CSV** DataFrame hasil resampling diekspor ke file `agregasi.csv` menggunakan `to_csv()`. Format CSV dipilih karena mudah dibuka di Excel maupun diolah lebih lanjut oleh tools lain. Index timestamp ikut tersimpan karena merupakan informasi penting dalam data time series.

Tahap 5 - Data Generator

Buat `data_generator.py` yang berjalan dalam infinite loop, menyisipkan data sensor acak setiap 2 detik.

Kode sebagai berikut:

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

pipeline = [
    { "$group": {
        "_id": "$mesin",
        "rata_suhu": { "$avg": "$suhu" },
        "max_getaran": { "$max": "$getaran" },
        "count": { "$sum": 1 }
    } },
    { "$sort": { "rata_suhu": -1 } }
]

results = collection.aggregate(pipeline)
print("Rata-rata suhu per mesin:")
for doc in results:
    print(f"{doc['_id']} -> rata:{doc['rata_suhu']:.2f}°C, max getaran:{doc['max_getaran']},  
jumlah data:{doc['count']}")

client.close()
```

Skrip ini berperan penting dalam mengisi database dengan data yang terus bertambah, sehingga dapat mensimulasikan kondisi produksi yang sebenarnya. Penggunaan `time.sleep(2)` berfungsi untuk mengatur jeda pengiriman data setiap 2 detik. Setelah skrip dijalankan, Anda dapat membuka Compass, melakukan refresh, dan mengamati dokumen yang muncul secara berkala tiap 2 detik. Proses ini dapat dihentikan kapan saja dengan menekan Ctrl+C.

Tahap 5B (Lanjutan) – Simulasi Data Sensor Melalui MQTT

Anda akan menjalankan dua skrip pada terminal yang berbeda: satu berperan sebagai publisher (sensor) dan satu lagi sebagai subscriber (penyimpan data). Pastikan broker MQTT sudah tersedia, baik menggunakan layanan publik seperti test.mosquitto.org maupun broker lokal seperti Mosquitto.

Pastikan anda sudah menginstall mqtt dengan mengetik di terminal:

```
pip install paho-mqtt
```

Pastikan juga anda telah menginstal matplotlib dengan mengetik di terminal:

```
pip install matplotlib
```

1. Baru anda buat file mqtt_publisher.py dan masukkan kode berikut:

```
import json, time, random
import paho.mqtt.client as mqtt
from datetime import datetime

BROKER = "test.mosquitto.org"
PORT = 1883
TOPIC = "pabrik/sensor/suhu"

client = mqtt.Client()
client.connect(BROKER, PORT, 60)

while True:
    data = {
        "mesin": f"CNC-{random.randint(1,5):02d}",
        "suhu": round(random.uniform(60, 100), 2),
        "getaran": round(random.uniform(0.1, 0.5), 2),
        "timestamp": datetime.utcnow().isoformat()
    }
    payload = json.dumps(data)
    client.publish(TOPIC, payload)
    print(f"[PUB] {payload}")

    time.sleep(2)
```

2. Buat file mqtt_subscriber.py dan masukkan kode berikut:

```

import json, os
import paho.mqtt.client as mqtt
from pymongo import MongoClient
from datetime import datetime
from dotenv import load_dotenv

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")
DB_NAME = os.getenv("DB_NAME")

mongo_client = MongoClient(MONGO_URI)
db = mongo_client[DB_NAME]
collection = db["sensor"]

def on_connect(client, userdata, flags, rc):
    print("Terhubung ke MQTT broker")
    client.subscribe("pabrik/sensor/suhu")

def on_message(client, userdata, msg):
    try:
        payload = json.loads(msg.payload.decode())
        payload['timestamp'] = datetime.fromisoformat(payload['timestamp'])
        result = collection.insert_one(payload)
        print(f"[MONGO] Tersimpan {result.inserted_id} - {payload['mesin']} {payload['suhu']}°C")
    except Exception as e:
        print("Error:", e)

mqtt_client = mqtt.Client()
mqtt_client.on_connect = on_connect
mqtt_client.on_message = on_message
mqtt_client.connect("test.mosquitto.org", 1883, 60)

mqtt_client.loop_forever()

```

Penjelasan:

- Import dan Koneksi MongoDB Library json digunakan untuk parsing payload MQTT, paho.mqtt.client sebagai client MQTT, dan pymongo untuk menyimpan data ke MongoDB. Koneksi ke MongoDB dibuat di awal program menggunakan URI dari .env, lalu koleksi sensor disiapkan sebagai tujuan penyimpanan data.

- Callback `on_connect` Fungsi ini dipanggil secara otomatis oleh library `paho-mqtt` ketika koneksi ke broker berhasil. Di dalamnya, client langsung melakukan subscribe ke topik `"pabrik/sensor/suhu"`. Alasan subscribe dilakukan di sini bukan di luar fungsi adalah untuk memastikan subscribe hanya terjadi setelah koneksi benar-benar tersambung, sehingga tidak ada pesan yang terlewat.
- Callback `on_message` Fungsi ini dipanggil otomatis setiap kali ada pesan masuk dari topik yang di-subscribe. Payload yang diterima berupa bytes, lalu di-decode menjadi string dan di-parse dari format JSON menjadi dictionary Python menggunakan `json.loads()`. Field timestamp yang semula berupa string ISO format dikonversi menjadi objek `datetime` menggunakan `datetime.fromisoformat()` agar tersimpan sebagai tipe `datetime` di MongoDB, bukan string. Dokumen kemudian disimpan ke koleksi `sensor` menggunakan `insert_one()`.
- Inisialisasi MQTT Client dan `loop_forever()` Objek `mqtt_client` dibuat, lalu dua callback di atas didaftarkan ke dalamnya. Setelah `connect()` dipanggil untuk menyambung ke broker, `loop_forever()` menjalankan event loop secara blocking program tidak akan berhenti dan terus memantau pesan yang masuk. Setiap kali ada pesan baru dari publisher, `on_message` langsung dipanggil secara otomatis.

Tahap 6 – Error Handling dan Logging

Tambahkan blok `try-except` pada operasi – operasi kritis di setiap skrip, dan konfigurasi logging ke dalam file dan konsol.

Di dalam file `crud_sensor.py` masukkan blok `try-except` pada operasi – operasi kritis di setiap skrip, lalu konfigurasikan logging ke file dan konsol.

Dengan kode lengkap sebagai berikut:

```
from crud_sensor import collection, client
from datetime import datetime, timedelta
import random
import logging

logging.basicConfig(
```

```

        level=logging.INFO,
        format='%(asctime)s - %(levelname)s - %(message)s',
        handlers=[
            logging.FileHandler("crud_sensor.log"),
            logging.StreamHandler()
        ]
    )

# Insert Many
data = []
for i in range(10):
    doc = {
        "mesin": f"CNC-{random.randint(1,3):02d}",
        "suhu": round(random.uniform(60, 100), 2),
        "getaran": round(random.uniform(0.1, 0.5), 2),
        "timestamp": datetime.utcnow() - timedelta(minutes=i*5),
        "status": "normal"
    }
    data.append(doc)

try:
    result = collection.insert_many(data)
    logging.info(f"Insert berhasil, {len(result.inserted_ids)} dokumen")
except Exception as e:
    logging.error(f"Insert gagal: {e}")

# Read
try:
    cursor = collection.find(
        {"mesin": "CNC-01"},
        {"_id": 0, "suhu": 1, "timestamp": 1}
    ).sort("timestamp", -1).limit(5)
    logging.info("Query CNC-01 berhasil")
    for doc in cursor:
        print(doc)
except Exception as e:
    logging.error(f"Query gagal: {e}")

# Update
try:
    update_result = collection.update_many(
        {"suhu": {"$gt": 90}},
        {"$set": {"status": "maintenance"}}
    )

```

```

    )
    logging.info(f"Update berhasil, {update_result.modified_count} dokumen diubah")
except Exception as e:
    logging.error(f"Update gagal: {e}")

# Delete
try:
    satu_jam_lalu = datetime.utcnow() - timedelta(hours=1)
    delete_result = collection.delete_many({"timestamp": {"$lt": satu_jam_lalu}})
    logging.info(f"Delete berhasil, {delete_result.deleted_count} dokumen dihapus")
except Exception as e:
    logging.error(f"Delete gagal: {e}")

client.close()
logging.info("Koneksi MongoDB ditutup")

```

Output:

```

Delete: 0 dokumen dihapus
[Done] exited with code=0 in 2.425 seconds

```

Penjelasan:

Dalam query ini terdapat tambahan import logging untuk semacam jejak kronologis kejadian. Ketika program berjalan di latar belakang, maka Anda dapat mendiagnosis masalah dalam program.

Latihan Mandiri

Latihan 1 – Impor Data Maintenance dari CSV

- Buat folder baru dengan nama latihan 6, lalu Anda ketikkan ini di terminal anda:

```
python -m venv venv
```

- Anda buat file .env dengan isi sebagai berikut

```

MONGO_URI=mongodb://localhost:27017
DB_NAME=latihan_6

```

- Lalu Anda buat file maintenance.csv dengan isi file seperti berikut:

```

mesin,tanggal,biaya,teknisi
CNC-01,2024-01-15,1200000,Budi
CNC-01,2024-01-28,800000,Andi
CNC-02,2024-01-10,500000,Citra
CNC-02,2024-02-05,1500000,Andi
LATHE-01,2024-02-18,950000,Budi
LATHE-01,2024-03-03,1100000,Citra
CNC-01,2024-03-20,300000,Eko
LATHE-02,2024-03-25,1800000,Eko

```

- Selanjutnya Anda buat file `import_maintenance.py` dengan query seperti berikut:

```

import pandas as pd
from pymongo import MongoClient

client = MongoClient('mongodb://localhost:27017/')
db = client['latihan_6']

df = pd.read_csv('maintenance.csv')
df["tanggal"] = pd.to_datetime(df["tanggal"])
data = df.to_dict(orient='records')

db["maintenance"].insert_many(data)

print("Import selesai:", len(data), "dokumen dimasukkan.")

```

- Terakhir anda buat file `query_maintenance.py` dengan query seperti berikut:

```

import pandas as pd
from pymongo import MongoClient

client = MongoClient('mongodb://localhost:27017/')
db = client['latihan_6']

# A. Find biaya > 1.000.000
hasil = db["maintenance"].find({"biaya": {"$gt": 1000000}})
df_hasil = pd.DataFrame(list(hasil))
print("=== Dokumen dengan biaya > 1.000.000 ===")
print(df_hasil[["mesin", "tanggal", "biaya", "teknisi"]])

# B. Update teknisi

```



```

db["maintenance"].update_one(
    {"mesin": "CNC-01", "biaya": 1200000},
    {"$set": {"teknisi": "Dewi"}}
)
print("\nUpdate selesai.")

# C. Agregasi per bulan
pipeline = [
    {
        "$group": {
            "_id": {
                "bulan": {"$dateToString": {"format": "%Y-%m", "date": "$tanggal"}}
            },
            "total_biaya": {"$sum": "$biaya"}
        }
    },
    {"$sort": {"_id.bulan": 1}}
]
hasil_agg = list(db["maintenance"].aggregate(pipeline))
df_agg = pd.DataFrame(hasil_agg)
df_agg["bulan"] = df_agg["_id"].apply(lambda x: x["bulan"])
df_agg = df_agg[["bulan", "total_biaya"]]
print("\n=== Total Biaya per Bulan ===")
print(df_agg)

```

Output:

```

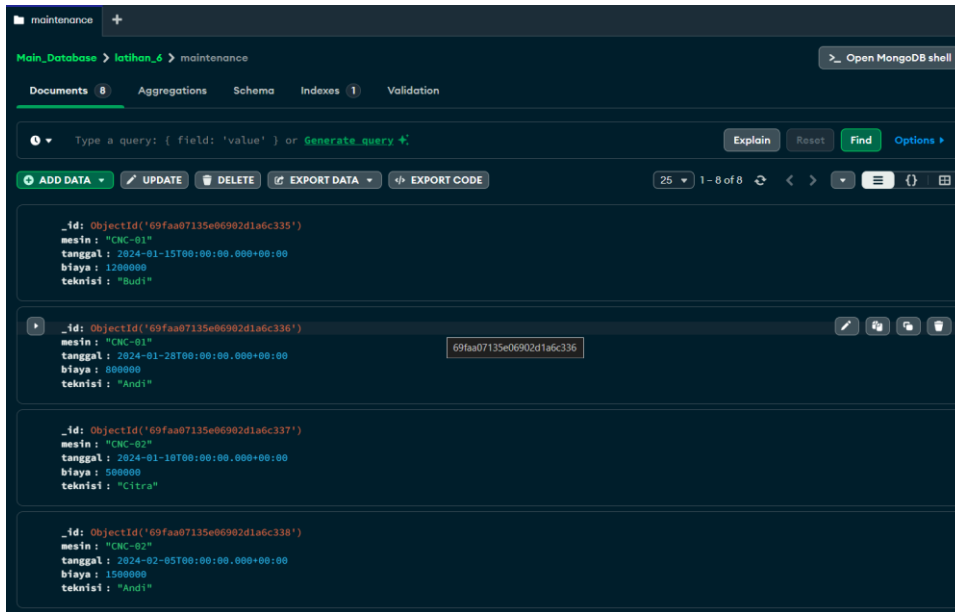
=== Dokumen dengan biaya > 1.000.000 ===
|  | mesin   tanggal   biaya   teknisi
0  | CNC-01  2024-01-15  1200000  Budi
1  | CNC-02  2024-02-05  1500000  Andi
2  | LATHE-01 2024-03-03  1100000  Citra
3  | LATHE-02 2024-03-25  1800000  Eko

Update selesai.

=== Total Biaya per Bulan ===
|  | bulan   total_biaya
0  | 2024-01  2500000
1  | 2024-02  2450000
2  | 2024-03  3200000

```

Hasil dari semua langkah tadi akan masuk ke database mongodb seperti berikut:



Latihan 2 – MQTT Data Produksi

- Buat file baru mqtt_pub_latihan.py di folder yang sama saja dengan query seperti berikut:

```
import time
import json
import random
from datetime import datetime

import paho.mqtt.client as mqtt

# Konfigurasi broker (bisa ganti kalau pakai lokal)
BROKER = "broker.hivemq.com"
PORT = 1883
TOPIC = "pabrik/produksi"

# Data referensi untuk random
batch_list = ["B001", "B002", "B003", "B004", "B005"]
mesin_list = ["M1", "M2", "M3", "M4"]

# Callback saat konek
def on_connect(client, userdata, flags, rc):
    if rc == 0:
```

```

        print("Connected to broker")
    else:
        print(f"Connection failed with code {rc}")

client = mqtt.Client()
client.on_connect = on_connect

client.connect(BROKER, PORT, 60)

client.loop_start()

try:
    while True:
        data = {
            "timestamp": datetime.now().isoformat(),
            "batch": random.choice(batch_list),
            "mesin": random.choice(mesin_list),
            "jumlah": random.randint(100, 500),
            "reject": random.randint(0, 50)
        }

        payload = json.dumps(data)
        client.publish(TOPIC, payload)

        print(f"Sent: {payload}")

        time.sleep(3)

except KeyboardInterrupt:
    print("Stopped by user")

finally:
    client.loop_stop()
    client.disconnect()

```

- Buat file satu lagi mqtt_sub_latihan.py dengan query seperti berikut:

```

import json
from datetime import datetime

import paho.mqtt.client as mqtt
from pymongo import MongoClient

BROKER = "broker.hivemq.com"

```

```

PORT = 1883
TOPIC = "pabrik/produksi"

MONGO_URI = "mongodb://localhost:27017/"
DB_NAME = "latihan_6"
COLLECTION_NAME = "produksi_mqtt"

mongo_client = MongoClient(MONGO_URI)
db = mongo_client[DB_NAME]
collection = db[COLLECTION_NAME]

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("Connected to MQTT broker")
        client.subscribe(TOPIC)
    else:
        print(f"Failed to connect, code {rc}")

def on_message(client, userdata, msg):
    try:
        payload = msg.payload.decode()
        data = json.loads(payload)

        # Ambil field dari publisher
        jumlah = data.get("jumlah", 0)
        reject = data.get("reject", 0)

        # Hindari division by zero
        if jumlah > 0:
            reject_rate = (reject / jumlah) * 100
        else:
            reject_rate = 0

        # Tambah field baru
        data["timestamp_terima"] = datetime.now()
        data["reject_rate"] = reject_rate

        # Cek peringatan
        if reject_rate > 5:
            print(f" WARNING: Reject rate tinggi! ({reject_rate:.2f}%)")
            data["peringatan"] = True
        else:
            data["peringatan"] = False

```

```

        # Simpan ke MongoDB
        collection.insert_one(data)

        print(f"Saved to MongoDB: {data}")

    except Exception as e:
        print(f"Error processing message: {e}")

# Setup Client MQTT
client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message

client.connect(BROKER, PORT, 60)

print("Subscriber berjalan... tekan Ctrl+C untuk berhenti")

try:
    client.loop_forever()
except KeyboardInterrupt:
    print("Stopped by user")
finally:
    client.disconnect()
    mongo_client.close()

```

Barulah anda jalankan di terminal dari my_pub_latihan dulu, lalu my_sub_latihan dari terminal dengan menuliskan seperti berikut:

```

python mqg_pub_latihan.py
python mqg_sub_latihan.py

```

Output Pub:

```

WARNING: Reject rate tinggi! (13.92%)
Saved to MongoDB: {'timestamp': '2026-05-06T09:54:01.688483', 'batch': 'B004', 'mesin': 'M1', 'jumlah': 316, 'reject': 44, 'timestamp_terima': datetime.datetime(2026, 5, 6, 9, 54, 9, 565914), 'reject_rate': 13.924050632911392, 'peringatan': True, '_id': ObjectId('69faad51b7bbcd156a75b5e')}
WARNING: Reject rate tinggi! (14.29%)
Saved to MongoDB: {'timestamp': '2026-05-06T09:54:04.689224', 'batch': 'B003', 'mesin': 'M1', 'jumlah': 336, 'reject': 48, 'timestamp_terima': datetime.datetime(2026, 5, 6, 9, 54, 9, 584668), 'reject_rate': 14.285714285714285, 'peringatan': True, '_id': ObjectId('69faad51b7bbcd156a75b5f')}
WARNING: Reject rate tinggi! (27.78%)
Saved to MongoDB: {'timestamp': '2026-05-06T09:54:07.690278', 'batch': 'B003', 'mesin': 'M4', 'jumlah': 162, 'reject': 45, 'timestamp_terima': datetime.datetime(2026, 5, 6, 9, 54, 9, 585806), 'reject_rate': 27.77777777777778, 'peringatan': True, '_id': ObjectId('69faad51b7bbcd156a75b60')}
WARNING: Reject rate tinggi! (12.02%)
Saved to MongoDB: {'timestamp': '2026-05-06T09:54:10.691172', 'batch': 'B003', 'mesin': 'M2', 'jumlah': 233, 'reject': 28, 'timestamp_terima': datetime.datetime(2026, 5, 6, 9, 54, 10, 987864), 'reject_rate': 12.017167381974248, 'peringatan': True, '_id': ObjectId('69faad52b7bbcd156a75b61')}
WARNING: Reject rate tinggi! (7.53%)
Saved to MongoDB: {'timestamp': '2026-05-06T09:54:13.691718', 'batch': 'B004', 'mesin': 'M3', 'jumlah': 385, 'reject': 29, 'timestamp_terima': datetime.datetime(2026, 5, 6, 9, 54, 13, 907595), 'reject_rate': 7.532467532467532, 'pe

```

Output Sub:

```
Sent: {"timestamp": "2026-05-06T09:54:10.691172", "batch": "B003", "mesin": "M2", "jumlah": 233, "reject": 28}
Sent: {"timestamp": "2026-05-06T09:54:13.691718", "batch": "B004", "mesin": "M3", "jumlah": 385, "reject": 29}
Sent: {"timestamp": "2026-05-06T09:54:16.692310", "batch": "B003", "mesin": "M1", "jumlah": 476, "reject": 7}
Sent: {"timestamp": "2026-05-06T09:54:19.692978", "batch": "B003", "mesin": "M1", "jumlah": 419, "reject": 40}
Sent: {"timestamp": "2026-05-06T09:54:22.693654", "batch": "B002", "mesin": "M1", "jumlah": 301, "reject": 7}
Sent: {"timestamp": "2026-05-06T09:54:25.694316", "batch": "B005", "mesin": "M3", "jumlah": 428, "reject": 19}
Sent: {"timestamp": "2026-05-06T09:54:28.695372", "batch": "B003", "mesin": "M2", "jumlah": 255, "reject": 20}
Sent: {"timestamp": "2026-05-06T09:54:31.696445", "batch": "B001", "mesin": "M4", "jumlah": 213, "reject": 12}
Sent: {"timestamp": "2026-05-06T09:54:34.697423", "batch": "B003", "mesin": "M4", "jumlah": 463, "reject": 34}
Sent: {"timestamp": "2026-05-06T09:54:37.698573", "batch": "B005", "mesin": "M3", "jumlah": 232, "reject": 9}
Sent: {"timestamp": "2026-05-06T09:54:40.699438", "batch": "B003", "mesin": "M4", "jumlah": 244, "reject": 37}
Sent: {"timestamp": "2026-05-06T09:54:43.700384", "batch": "B004", "mesin": "M1", "jumlah": 378, "reject": 24}
Sent: {"timestamp": "2026-05-06T09:54:46.701119", "batch": "B002", "mesin": "M1", "jumlah": 369, "reject": 38}
Sent: {"timestamp": "2026-05-06T09:54:49.701870", "batch": "B001", "mesin": "M2", "jumlah": 362, "reject": 40}
Sent: {"timestamp": "2026-05-06T09:54:52.702512", "batch": "B002", "mesin": "M1", "jumlah": 102, "reject": 38}
```

KESIMPULAN

Kesimpulan dari praktikum ini adalah bahwa penggunaan Python sebagai perantara dalam memasukkan data dummy ke dalam database MongoDB memberikan proses yang lebih fleksibel, terstruktur, dan mudah dikembangkan dibandingkan metode statis seperti impor langsung dari file JSON. Dengan memanfaatkan Python, proses integrasi data dapat dilakukan secara otomatis sekaligus memungkinkan adanya manipulasi dan validasi data sebelum disimpan. Hal ini membuat simulasi pengolahan data menjadi lebih mendekati kondisi nyata di lingkungan produksi serta membantu memahami bagaimana berbagai komponen teknologi dapat saling terintegrasi dalam sistem berbasis data.